

Subgraphs A Subgraph is essentially a graph that is embedded and executed as a single node inside another larger parent graph.

## → The need for Subgraphs (Multi-Agent Systems)

As GenAI applications scale, (e.g. building an entire software development AI), putting all the logic (planning, backend coding, frontend coding, testing, DevOps) into a single graph becomes incredibly complex. By breaking the system down into smaller, specialized agents (where each agent is subgraph with its own tools, memory, guardrails), we can create a manageable multi-agent architecture.

## \* Core benefits of Subgraphs:

(a) Modularity, Reusability and Maintainability

(b) Failure Isolation

(c) State Separation: Instead of all agents clashing over a single massive state dictionary, each subgraph can maintain its own isolated state.

(d) Granular Observability: Using LangSmith, we can trace and evaluate the specific performance, token usage, and latency of individual subgraphs rather than just the parent graph.

## → Two Mechanisms for Implementation:-

1. Invoking from inside a node ⇒ We create two completely independent graphs. In the parent graph, a standard node contains a single line of execution that explicitly invokes the execution of subgraph.



In this setup, states are completely isolated between the parent and the child.

2. Adding the graph as a Node :- We add the compiled subgraph directly as a node in the parent graph. In this setup, the state is shared; the subgraph operates using the parent's state keys.

## Failure Isolation in Subgraphs

By default, in Lang Graph if a subgraph raises an unhandled Python exception, it will bubble up and crash the entire parent execution. To achieve true failure isolation, we must implement explicit `.with_fallbacks()` logic on the node, or use standard `try/except` blocks within the subgraph's nodes to gracefully handle the error and update the state safely.

## Memory In LLMs / Langgraph

Large Language Models (LLMs) are stateless mathematical functions at inference, represented as  $y = f(x; \theta)$ , where  $\theta$  represents the billions of parameters set during training,  $x$  represents the input token prompts, and  $y$  represents the generated output tokens. Because the mathematical function is stateless, the output  $y_2$  depends only on current input  $x_2$ , with no intrinsic memory of the prior inputs ( $x_1$ ) or output ( $y_1$ ). To construct a conversational interface, we must create a conversation buffer (short-term memory) where the entire chat history is concatenated and passed to the model on every subsequent invocation.

## Short-Term Memory (STM) & Thread-Scoped Persistence

In modern-chatbot interfaces, short-term memory is bounded within a single conversation session, historically referred to as a "thread". A short-term memory is "thread-scoped", meaning it does not exist outside the existing thread.



Switching to a new conversation ~~creates~~ resets the message buffer, preventing token bloat and maintaining conversational focus.

**\*LangGraph Checkpointers:-** LangGraph manages Short-Term Memory using checkpointers. A chекpointer automatically serializes and saves the entire graph state at every "Superstep" (the boundary of node execution).

**\*In-Memory Vs Persistent Saving:-**

→ **MemorySaver** is an in-memory chекpointer that writes states directly to RAM. Because RAM is volatile, restarting the system or clearing the application server permanently erases all conversation history.

→ **PostgresSaver** For production systems, LangGraph suggests using a production level database to persist states. This is achieved using persistent checkpointers like those <sup>backed</sup> by PostgreSQL. This setup survives application restarts, crashes, and server migrations.

### Context Window Mitigation Techniques

Every LLM has a **context-window**, the maximum number of tokens it can read and remember in a single interaction before generating a response. If the accumulated conversation history exceeds this limit, the LLM's response quality degrades, leading to incoherence or hallucinations. Two primary techniques are used to mitigate this context overflow:

**A. Trimming (Trim messages)** ⇒ Trimming sets a maximum token limit for the inputs sent to the LLM. Before invoking the model, the system calculates the cumulative token count of the history. If it exceeds the limit, it retains only the most recent  $n$  messages that fit within the limit and discards older ones.

→ **State Preservation**: Crucially, Trimming does not delete the discarded messages from the actual graph state or the database. The history remains intact at the chекpointer, but the older messages are temporarily hidden from the LLM during that specific invocation to stay within token budgets.



→ **Inherent Flaw:** Trimming assumes that only the most recent messages contain relevant context. In complex, long-running tasks, vital details established early in the conversation are permanently lost to the LLM.

**B. Summarization and Delete** → Rather than discarding older messages, they are sent to a secondary LLM to generate a dense, concise summary. The prompt sent to the primary model then consists of the historical summary combined with the most recent  $n$  messages.

→ **Incremental Updates:** As the conversation progresses, the existing summary is continuously updated by merging it with the new blocks of old messages, keeping the overall token count highly stable and well within the context window.

→ **Explicit State Deletion:** To prevent confusion (where the model would receive both the raw messages and their summary), the summarized messages are permanently removed from the graph state using a deletion utility (such as `remove-messages`). This represents a physical mutation of the checkpoint state.

## Long Term Memory (LTM)

While short-term memory excels at holding immediate context within a single thread, its thread-scoped nature introduces 4 major limitations:

- (a) **No continuity between conversations:** It cannot carry over user-preferences (such as programming language preferences or output format choices) from one thread to another.
- (b) **No compounding knowledge:** Every new thread forces the user to retrain the agent from scratch, preventing the system's intelligence from evolving with the user.
- (c) **No cross thread reasoning:** The agent cannot answer questions about past interactions or compare current tasks with historical ones.
- (d) **No true personalization:** Building a true, evolving personal agent is



impossible, if its memory is strictly compartmentalised inside independent threads.

To solve these issues, Long Term Memory (LTM) operates outside of thread-boundaries, preserving selected extracted knowledge across separate sessions for a long period.

## Classification of Long Term Memory

- (a) Episodic Memory  $\Rightarrow$  Remembers what happened in past events, interactions and sequences. It allows the agent to recall past decisions, successes and failures to answer questions like: "How did we resolve this issue last time?"
- (b) Semantic Memory  $\Rightarrow$  Remembers facts about the users, system or environment. Examples include user demographic details, tool configurations, budget constraints, or technology preferences.
- (c) Procedural Memory  $\Rightarrow$  Remembers how to do things. It stores strategies, rules, preferred workflows, and learned behaviors custom-tailored to the user (e.g. avoiding sub-queries in database calls or adopting step-by-step reasoning formats).

## The Four-Step Long-Term Memory Workflow

Long Term Memory operates via a distinct, four-step lifecycle:

1. Creation/Update (Extraction)  $\Rightarrow$  During a conversation, the system continuously monitors users messages, model outputs and tool results. It filters out conversational noise and identifies memory candidates. It then decides whether to create a new memory, edit an existing record, or ignore the input.
2. Storage  $\Rightarrow$  The extracted memory is written to a durable store with metadata and identifiers such as (user or agent tags) to ensure it survives crashes and system restarts.
3. Retrieval  $\Rightarrow$  When a new conversation begins, the system selectively searches the database based on the active user's query. Rather than pulling the entire memory database, it performs a targeted retrieval of relevant facts,



4. Injection  $\Rightarrow$  The retrieved memories are pulled into the short-term memory buffer (the prompt) as context. The LLM views the data as standard input tokens, enabling it to customise its response without the user needing to restate their preferences.

## Storage Architectures and Namespaces in LangGraph

In LangGraph, Long-Term Memory is implemented using the Store architecture, governed by the BaseStore abstract class. BaseStore exposes core methods including get (fetch a single memory), search (query multiple memories), put (write/update a memory), and delete.

\* Hierarchical Namespaces  $\Rightarrow$  To keep memories organized, the store uses namespaces, which acts like directory structures or folders within a storage drive. Namespaces are structured as tuples of strings - for instance ("users", "U1", "preferences"). This allows complete partition of user data, ensuring User 1's profile never leaks into User 2's session.

\* Semantic Search Capability  $\Rightarrow$  When creating a store, we can configure it with an Embedding Model. During a search operation, the user's current query is extracted / converted into an embedding vector, which is compared against the stored memories to retrieve the most semantically relevant items, even if they do not match the keywords exactly.